Prediction Model for Adverse Drug Reactions Using Deep Learning Methods

A Final Technical Report Submitted for
DS 5500: Capstone — Applications in Data Science

Authors: Elizabeth Coquillette
Hithaishi Raghavendra Reddy
Ishan Chotalia

Team: Team 9

Instructor: Dr. Fatema Nafa

Course: DS 5500 — Capstone: Applications in Data Science

Institution: Northeastern University, Khoury College of Computer Sciences

Date: April 6, 2026

GitHub Repo: Project Repository

Abstract

Background: Adverse drug reactions (ADRs) are a major cause of preventable morbidity and hospital admissions worldwide, yet existing approaches lack patient-specific risk prediction.
Objective: This project aims to develop a predictive model that estimates the likelihood of an adverse drug reaction for individual patients using electronic health record data.
Methods: We use the MIMIC-IV clinical database and implement both classical machine learning models and deep learning architectures including multilayer perceptrons, ResNet-style models, and attention-based networks.
Results: Deep learning models outperform baseline approaches, achieving higher AUROC and F1 scores in predicting ADR risk, particularly when incorporating expanded clinical features.
Conclusion: Patient-specific ADR prediction is feasible using structured clinical data and deep learning, with potential to improve clinical decision-making and patient safety.

Keywords: adverse drug reactions, MIMIC-IV, deep learning, healthcare AI, risk prediction

Contents

List of Figures

List of Tables

1. Introduction

1.1 Problem Statement

Adverse drug reactions (ADRs) represent a significant challenge in modern healthcare, contributing to increased morbidity, hospital admissions, and healthcare costs[Pirmohamed et al., 2004]. While medications are prescribed based on general clinical guidelines, the risk of adverse reactions varies significantly across individuals due to differences in physiology, comorbidities, and concurrent medications.

Current clinical practice lacks tools that provide personalized estimates of ADR risk. Most available information is based on population-level statistics rather than patient-specific data. As a result, clinicians often make prescribing decisions without fully understanding individualized risk, leading to preventable adverse outcomes.

This project addresses the problem of predicting adverse drug reactions at the patient level using electronic health record (EHR) data. By leveraging machine learning and deep learning [LeCun et al., 2015] techniques, we aim to provide a predictive model that can estimate ADR risk based on a patient's clinical profile.

1.2 Motivation and Significance

ADRs are estimated to account for approximately 7% of hospital admissions globally and impose an annual cost exceeding $136 billion in the United States. Early prediction of ADRs has the potential to significantly reduce both clinical and economic burden.

The increasing availability of large-scale EHR datasets such as MIMIC-IV enables the application of data-driven approaches to healthcare problems. Machine learning and deep learning methods can identify complex patterns in patient data that are not easily captured by traditional rule-based systems.

This project aims to bridge the gap between data availability and clinical decision support by building an end-to-end predictive system for ADR risk.

1.3 Research Questions / Objectives

1. Can patient-specific clinical features be used to accurately predict adverse drug reactions?
2. Do deep learning models outperform classical machine learning approaches for ADR prediction?
3. How does feature engineering impact predictive performance in clinical datasets?

1.4 Report Structure

This report is organized as follows: Section 2 reviews related work; Section 3 describes the data pipeline; Section 4 details model selection and implementation; Section 5 presents results and evaluation; Section 6 covers testing and validation; Section 7 documents version control and team contributions; Section 8 addresses deployment and reproducibility; and Section 10 concludes with future work.

2. Related Work

2.1 EHR-Based ADR Prediction Using Classical Machine Learning

Early computational approaches to ADR prediction relied on classical machine learning applied to structured EHR data. Zhao et al. [2021] trained SVM, Random Forest, AdaBoost, XGBoost, and a shallow ANN on an imbalanced Chinese electronic medical record dataset, using NLP to convert unstructured clinical notes into numeric features. They found that ensemble methods—particularly AdaBoost and XGBoost—consistently outperformed standalone classifiers on imbalanced data, and that ANN provided competitive performance without explicit ensemble construction. However, their study was limited to a single-center Chinese hospital dataset, relied on binary classification for a single ADR outcome, and did not investigate the effect of expanding clinical features.

Hu et al. [2024] conducted a systematic review and meta-analysis of ten studies applying ML to multi-ADR prediction from EHRs, reporting a mean AUROC of 0.76 across studies, with Random Forest combined with resampling-based approaches achieving the highest AUCs (0.94–0.95). Their review highlighted that RF is the most commonly used algorithm in this space and that class imbalance remains the dominant practical challenge. Critically, no included study deployed its models as an interactive clinical tool or combined multiple deep learning architectures for systematic comparison.

Farnoush et al. [2024] used FAERS (FDA Adverse Event Reporting System) data to develop RF and deep learning models for multi-label ADR classification across 30 ADR types, integrating chemical, molecular, biological, and demographic features from DrugBank and PubChem. They found that including demographic features (age and gender) alongside drug molecular features improved both AUC and mean average precision. Unlike our work, they relied on post-marketing pharmacovigilance reports rather than real-time clinical EHR data, making their approach retrospective and dependent on external drug databases unavailable at the point of prescribing.

Yasrebi-de Kom et al. [2023] reviewed EHR-based prediction models for in-hospital adverse drug events, emphasizing that ICD codes are the most common labeling mechanism but are prone to under-reporting due to inconsistent coding practices across institutions. This limitation directly

motivated our use of a custom `adr_labels.csv` mapping to standardize label extraction from MIMIC-IV diagnosis tables.

2.2 Deep Learning Architectures for Clinical Prediction

Several deep learning architectures have been adapted for EHR-based clinical prediction, each with different strengths on tabular and sequential data.

Fridgeirsson et al. [2023] benchmarked four attention-based deep learning models—a recurrent neural network, a transformer with and without reverse distillation, and a graph neural network—against logistic regression and XGBoost on a large general practitioner EHR database. They found that deep learning improved discrimination by up to 2.5% in AUROC and 7.4% in AUPRC over classical models, but that gains varied substantially by task, and that attention-based models were not uniformly superior. Their work directly motivates our use of an attention-based classifier as one of four architectures in a systematic comparison.

Li et al. [2019] proposed a deep neural network that jointly leverages chemical structure, biological activity, and biomedical text embeddings of drugs for ADR detection, achieving strong performance on the SIDER side-effect database. Their approach required rich external drug databases; our work instead derives all features from the EHR itself, making predictions possible without external pharmaceutical data sources.

Huang et al. [2023] introduced pADR, a personalized ADR prediction framework that fuses patient EHR records with drug chemical information through a hierarchical multi-sourced transformer. pADR outperformed drug-feature-only baselines by modeling complex inter-source interactions. In contrast to pADR, our approach is entirely EHR-derived and does not require chemical representations of drugs, making it more directly deployable in a clinical setting where molecular data is rarely available at prescription time.

Li et al. [2020] introduced BEHRT, a BERT-style transformer pretrained on longitudinal EHR sequences from 1.6 million patients for multi-disease diagnosis prediction, achieving 8–13% improvement in average precision over state-of-the-art deep EHR models. BEHRT treats each clinical visit as a token, enabling sequence-level attention across a patient's history. Our attention model operates over a single prescription event rather than a longitudinal sequence—a deliberate design choice given our tabular, non-temporal feature representation—and represents a simpler but clinically deployable instantiation of the attention idea.

Yang et al. [2023] proposed TransformEHR, a generative encoder-decoder transformer pretrained on MIMIC data to predict future disease outcomes from past visit sequences, improving AUPRC by up to 24% on challenging prediction tasks. Their model requires longitudinal multi-visit sequences, whereas our approach produces a prediction at the moment of a single drug prescription event—a practically important distinction for real-time clinical deployment.

2.3 Datasets, Benchmarks, and Evaluation Practices

The MIMIC dataset family has become the de facto standard for clinical ML research. Johnson et al. [2023] describe MIMIC-IV, a publicly available, de-identified EHR database sourced from BIDMC covering over 50,000 patients and 76,000 ICU admissions between 2008 and 2019. MIMIC-IV provides structured tables including patient demographics, diagnoses (ICD-9 and ICD-10), prescriptions, lab events, and ICU stays, which we use directly as our primary data source.

Gupta et al. [2022] provide an open-source preprocessing pipeline for MIMIC-IV that standardizes data extraction, cleaning, and imputation for downstream ML tasks. Their pipeline highlights the significant preprocessing challenges inherent to MIMIC-IV—outlier removal, missing value imputation, temporal alignment—that we address in our own `preprocessing.py` implementation.

Hu et al. [2023] provide a comprehensive survey of machine-learning-based ADE risk prediction from observational health data, emphasizing that AUROC alone is insufficient for imbalanced ADR tasks. They recommend AUPRC as a complementary metric that more accurately reflects model performance on the positive (ADR) class, and advocate for evaluating models at multiple decision thresholds rather than at a single cut-off. Our evaluation framework directly adopts these recommendations, reporting AUROC, AUPRC, sensitivity, and specificity at thresholds of 0.3, 0.5, and 0.7.

Lin et al. [2017] introduced Focal Loss, originally for dense object detection, as a solution to extreme class imbalance. By adding a modulating factor $(1-p_t)^\gamma$ to binary cross-entropy, Focal Loss reduces the contribution of easy, well-classified examples and focuses training on hard minority-class cases. With $\gamma = 2$ and $\alpha = 0.25$ (the same values we adopt), it has since been widely applied to medical classification tasks where positive events are rare [Yeung et al., 2022].

2.4 Positioning of This Work

Our project occupies a distinct position in the ADR prediction literature along several dimensions.

EHR-only, prescription-event-level prediction. Prior deep learning work on ADRs has typically relied on external drug databases [Li et al., 2019, Farnoush et al., 2024], longitudinal visit sequences [Li et al., 2020, Yang et al., 2023], or post-marketing pharmacovigilance reports [Farnoush et al., 2024]. We instead predict ADR risk from a single prescription event using only features available in the EHR at the point of prescribing—demographics, drug metadata, treatment context, and clinical lab values—making the approach applicable in real time without external data dependencies.

Systematic multi-architecture comparison. While individual papers typically propose and evaluate a single architecture, we train and compare four deep learning models (MLP, ResNet,

Attention, EmbeddingMLP) and a classical ML baseline under identical experimental conditions on the same MIMIC-IV data, enabling a controlled comparison of inductive biases for this specific prediction task.

Feature engineering as a primary contribution. Our key empirical finding—that expanding from 12 to 20 features by adding organ-function lab values (creatinine, ALT, AST) and comorbidity flags (renal and liver disease ICD indicators) improved AUROC by up to 0.29 across all models—highlights the primacy of clinical domain knowledge in feature design over architectural choice. This finding echoes Hu et al. [2024]'s observation that feature quality is the dominant predictor of performance in EHR-based ADR models.

Clinical deployment. Unlike most academic ADR prediction papers, we wrap our trained models in an interactive Streamlit dashboard that provides real-time risk scores, model performance comparisons, and session-level prediction logging—bridging the gap between research prototype and clinical decision support tool.

3. Data Pipeline and Preprocessing

3.1 Dataset Description

We use the MIMIC-IV Clinical Database (v2.2) [Johnson et al., 2023], a large, publicly available dataset containing de-identified electronic health records from Beth Israel Deaconess Medical Center.

The dataset includes approximately 300,000 patients and over 500,000 hospital admissions. Key tables used in this project include:

- **patients.csv**: demographic information such as age and gender
- **prescriptions.csv**: medication details including drug name, dose, and administration route
- **diagnoses_icd.csv**: diagnosis codes (ICD-9 and ICD-10)

The dataset provides longitudinal, time-stamped clinical data, enabling modeling of temporal relationships between drug exposure and subsequent outcomes.

3.2 Exploratory Data Analysis (EDA)

We performed exploratory data analysis to better understand the distribution of medications and diagnosis codes within the dataset.

Figure 1 shows the top 10 most frequently prescribed medications in the sampled dataset. Common drugs such as insulin, sodium chloride, and acetaminophen appear most frequently, reflecting typical hospital treatment patterns and validating the realism of the dataset.

Figure 2 presents the distribution of ICD diagnosis code versions. The dataset contains both ICD-9 and ICD-10 codes, with ICD-9 being more prevalent in the sampled data. This highlights the need to handle both coding systems when constructing ADR labels.

These exploratory insights guided our feature engineering and labeling strategy, particularly in selecting relevant medications and ensuring compatibility across diagnosis code systems.

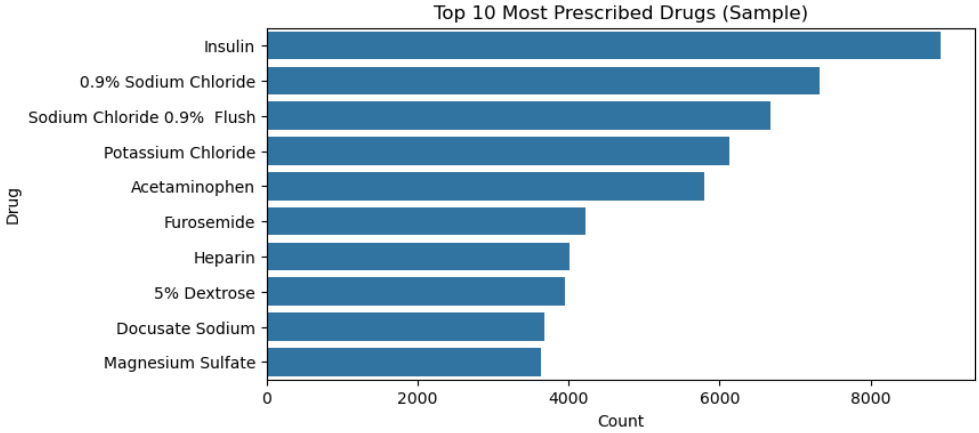

Figure 1: Top 10 most frequently prescribed drugs in the sampled MIMIC-IV dataset.

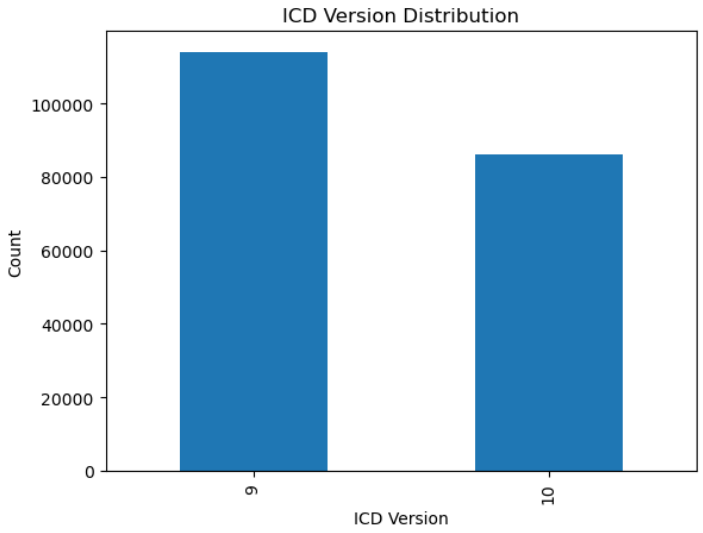

Figure 2: Distribution of ICD-9 and ICD-10 diagnosis codes in the dataset.

3.3 Data Cleaning and Preprocessing

The MIMIC-IV dataset contains real-world clinical data, which is inherently noisy, incomplete, and heterogeneous. To ensure data quality and model reliability, we performed several preprocessing steps including handling missing values, removing duplicates, correcting data types, and filtering relevant records.

Duplicate records at the patient-admission level were removed to avoid data leakage and bias. Timestamp fields such as prescription start and stop times were converted to datetime format to enable temporal reasoning. Additionally, irrelevant or low-frequency features were filtered to reduce dimensionality and improve model efficiency.

3.3.1 Missing Value Strategy

Missing values are common in electronic health records due to irregular clinical measurements. Instead of discarding large portions of data, we adopted a hybrid imputation strategy.

For numerical variables such as laboratory values (e.g., creatinine, ALT), missing values were imputed using the median, as it is robust to outliers. Additionally, binary missing indicators were created to preserve information about whether a value was originally absent.

For categorical variables such as drug route or gender, missing values were imputed using the mode. In cases where missingness itself carried clinical meaning, a separate "unknown" category was introduced.

This approach balances data retention with robustness and reflects realistic clinical scenarios where not all measurements are available for every patient.

3.3.2 Outlier Handling

Outliers in clinical data may represent either data errors or clinically significant extreme values. Therefore, outlier handling was performed carefully to avoid removing meaningful signals.

We used the Interquartile Range (IQR) method to identify extreme values in numerical features such as laboratory measurements. Values outside 1.5 times the IQR were flagged as potential outliers.

Instead of removing these observations, we applied clipping to cap extreme values within reasonable bounds. This preserves important clinical variation while preventing instability during model training.

3.4 Feature Engineering and Transformation

Feature engineering focused on extracting clinically meaningful signals from structured EHR data.

Core features included patient demographics (age, gender), medication attributes (drug name, dose, route), and treatment-related variables such as duration and frequency.

We expanded this feature set by incorporating clinically relevant indicators such as polypharmacy count (number of concurrent medications), disease flags derived from ICD codes (e.g., renal or liver conditions), and laboratory measurements including creatinine and liver enzymes.

Numerical features were standardized using StandardScaler to ensure consistent scaling across variables, particularly for deep learning models. Categorical variables were encoded using one-hot encoding, while high-cardinality features such as drug names were limited to the most frequent categories to reduce sparsity.

This combination of feature engineering and transformation enables the model to capture both patient state and treatment context, which are critical for predicting adverse drug reactions.

3.5 Train / Validation / Test Split

Table 1: Data partition sizes

Split	Samples	Percentage	Notes
Training	64% of dataset	64%	Stratified by ADR label
Validation	16% of dataset	16%	Used for hyperparameter tuning
Test	20% of dataset	20%	Held-out, never seen during training

The dataset was split using stratified sampling to preserve the distribution of ADR and non-ADR cases across all subsets. This is particularly important due to the class imbalance inherent in ADR prediction.

3.6 Pipeline Pseudo-code

Pseudo-code: End-to-End Data Preprocessing Pipeline

Require: Raw MIMIC-IV dataset $\mathcal{D}$, configuration file
Ensure: Cleaned and model-ready datasets $(\mathcal{D}_{train}, \mathcal{D}_{val}, \mathcal{D}_{test})$

Load patient, prescription, and diagnosis tables
Validate schema and required columns
Convert timestamp columns to datetime format
Remove duplicate records
Generate ADR labels using ICD-9 and ICD-10 codes
for each numerical feature f **do**
    Impute missing values using median
    Create missing indicator feature
    Apply StandardScaler
end for
for each categorical feature g **do**
    Impute missing values using mode
    Apply one-hot encoding
end for
Compute derived features (polypharmacy count, disease flags, lab summaries)
Perform stratified split: 64% train / 16% validation / 20% test
Save processed datasets for modeling
return $\mathcal{D}_{train}, \mathcal{D}_{val}, \mathcal{D}_{test}$

4. Model Selection and Implementation

4.1 Model Family Justification

Table 2: Model family comparison for this task

Family	Strengths	Limitations	Selected?
Classical ML	Interpretable, fast, strong tabular baseline	Limited in capturing complex interactions	✓
Deep Learning	Captures nonlinear interactions, scalable	Requires tuning and compute	✓
LLM / Foundation	Strong generalization in text tasks	Not suitable for structured tabular data	—

We selected both classical machine learning and deep learning approaches to evaluate performance trade-offs for structured clinical data. Logistic regression and tree-based models serve as strong baselines due to their interpretability and efficiency on tabular datasets.

Deep learning models were included to capture complex nonlinear interactions between patient features, medications, and clinical conditions. Given the size of the MIMIC-IV dataset and the structured nature of the features, feedforward architectures such as multilayer perceptrons (MLP), ResNet-style networks, and attention-based models were chosen.

Large language models (LLMs) were not considered appropriate for this task, as the input data is structured rather than unstructured text.

4.2 Model Architecture

We implemented multiple deep learning architectures using PyTorch:

Multilayer Perceptron (MLP): A fully connected neural network with multiple hidden layers, ReLU activations, and dropout for regularization. This serves as the primary deep learning baseline.

ResNet-style Model: A feedforward architecture with residual connections that allow gradients to propagate more effectively through deeper networks. This improves stability and performance on deeper architectures.

Attention-based Model: A lightweight attention mechanism applied to feature embeddings, enabling the model to dynamically weight important clinical features such as lab values or medication attributes.

All models take tabular feature vectors as input and output a probability score for adverse drug reaction classification using a sigmoid activation.

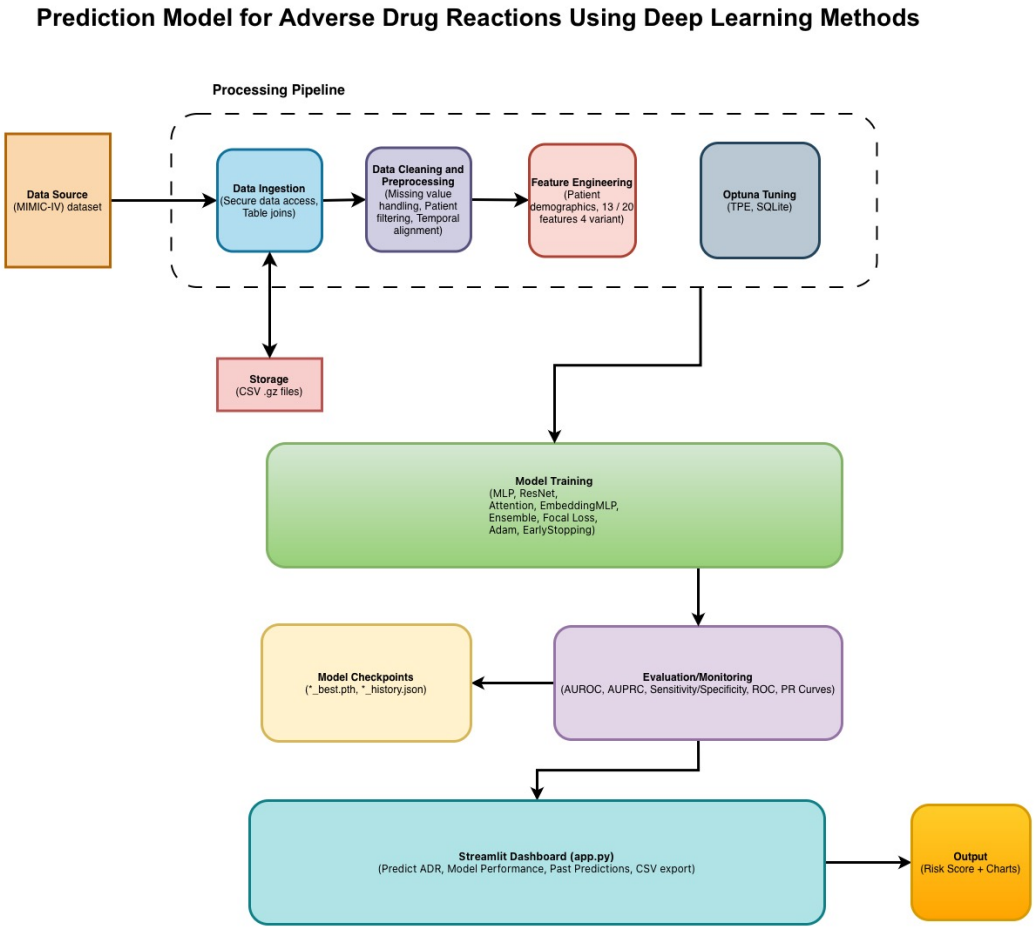

Figure 3: End-to-end architecture of the ADR prediction pipeline.

4.3 Coding Paradigm and Design Architecture

4.3.1 Paradigm Choice

The project follows a hybrid programming paradigm combining procedural and modular design. Core machine learning workflows such as training and evaluation are implemented procedurally for clarity and reproducibility, while reusable components such as data loading, preprocessing, and model definitions are modularized into separate scripts.

This approach balances readability with scalability, allowing rapid experimentation in notebooks while maintaining a clean and reusable codebase in the `src/` directory.

4.3.2 Module Structure

```
Prediction-Model-for-Adverse-Drug-Reactions/
|-- notebooks/
|   |-- 01_data_exploration.ipynb
|   |-- 02_adr_labeling.ipynb
|   |-- 03_baseline_models.ipynb
|
|-- src/
|   |-- data_loader.py        # Load MIMIC tables
|   |-- preprocessing.py      # Feature engineering + splits
|   |-- models.py             # Model definitions
|   |-- train.py              # Training loop
|   |-- train_MLP.py          # MLP-specific training
|   |-- tune_MLP.py           # Hyperparameter tuning
|   |-- tune_EMLP.py          # Advanced model tuning
|   |-- evaluate.py           # Evaluation metrics
|
|-- app.py                    # Streamlit dashboard
|-- requirements.txt
|-- README.md
```

Listing 1: Repository directory structure

4.4 Hyperparameters

Table 3: Hyperparameter configuration

Parameter	**Value**	**Tuning Method**
Learning rate	0.001	Optuna tuning
Batch size	64	Manual selection
Epochs	20	Early stopping (patience=5)
Optimizer	Adam	—
Dropout rate	0.3	Cross-validation
Hidden layers	2–4 layers	Optuna tuning

4.5 Training Procedure Pseudo-code

Pseudo-code: Model Training Loop

Require: Training set $\mathcal{D}_{train}$, validation set $\mathcal{D}_{val}$, model $\mathcal{M}$, hyperparameters θ
Ensure: Trained model $\mathcal{M}^*$

Initialize model weights $\mathcal{M}$ (random or pre-trained)
Set optimizer $\leftarrow$ Adam($lr = \theta.lr$, $\beta_1 = 0.9$, $\beta_2 = 0.999$)
Set $best\_val\_loss \leftarrow \infty$, $patience\_counter \leftarrow 0$
for epoch $= 1$ to $\theta.max\_epochs$ **do**
  $\mathcal{M}.train()$
  for each mini-batch (X_b, y_b) in $\mathcal{D}_{train}$ **do**
    $\hat{y}_b \leftarrow \mathcal{M}(X_b)$ {Forward pass}
    $\mathcal{L}_{train} \leftarrow$ Loss$(\hat{y}_b, y_b)$
    Backpropagate $\mathcal{L}_{train}$
    Clip gradients (max norm = 1.0)
    optimizer.step(); optimizer.zero_grad()
  end for
  $\mathcal{L}_{val} \leftarrow$ EVALUATE($\mathcal{M}$, $\mathcal{D}_{val}$)
  if $\mathcal{L}_{val} < best\_val\_loss$ **then**
    $best\_val\_loss \leftarrow \mathcal{L}_{val}$
    Save checkpoint: $\mathcal{M}^* \leftarrow \mathcal{M}$
    $patience\_counter \leftarrow 0$
  else
    $patience\_counter \mathrel{+}= 1$
  end if
  if $patience\_counter \geq \theta.patience$ **then**
    break {Early stopping}
  end if
end for
return $\mathcal{M}^*$

4.6 Fine-tuning / Prompt Engineering

This project does not involve large language models or prompt engineering. All models were trained from scratch using structured tabular data.

However, hyperparameter tuning was performed using Optuna to optimize model performance. Key parameters such as learning rate, network depth, and dropout were tuned using validation set performance.

No transfer learning was applied, as pre-trained models for structured EHR tabular data are not commonly available.

5. Results and Model Evaluation

5.1 Evaluation Metrics

Given the class imbalance in the dataset—approximately 17% of prescriptions are associated with an adverse drug reaction (ADR)—accuracy is a misleading evaluation metric. A trivial classifier that predicts the majority class would achieve over 80% accuracy while detecting no ADRs at all. We therefore prioritize two threshold-free metrics as our primary evaluation criteria:

- **AUROC** (Area Under the Receiver Operating Characteristic Curve): measures discriminative ability across all classification thresholds. A value of 0.5 indicates chance performance; 1.0 indicates perfect separation.
- **AUPRC** (Area Under the Precision-Recall Curve): particularly informative under class imbalance, as it focuses on performance over the minority (positive) class. A random classifier achieves AUPRC equal to the base rate (≈0.17).

Threshold-dependent metrics (sensitivity, specificity, precision, and F1) are also reported for interpretability, evaluated at a threshold of 0.5 for classical ML models and 0.3 for deep learning models (the latter chosen because DL models produced systematically low predicted probabilities, and a lower threshold better reflects their operating point on the precision-recall curve).

5.2 Baseline Comparison

All models were trained and evaluated on the *min1000* dataset—prescriptions for drugs with at least 1,000 observations—using the 19-feature set. We additionally report results using an expanded 20-feature set with the added variable `prior_adr`, which is a variable that disproportionately drives the model performance but is not useful in the real world, where patient history might be more sparse than in this dataset. To further demonstrate the role of the `prior_adr` variable, we also report results from a naïve baseline model that uses that single feature.

The Random Forest achieved the strongest performance with the 19 feature model, with AUROC = 0.8815 and AUPRC = 0.7520. Among deep learning architectures, the Embedding MLP performed best (AUROC = 0.7884, AUPRC = 0.5165), benefiting from dedicated embedding layers for categorical features such as drug identity, administration route, and dose unit. Notably, all DL

Table 4: Model performance on the held-out test set (19-feature set). ML models evaluated at threshold = 0.5; DL models at threshold = 0.3.

Model	Type	AUROC	AUPRC	Sensitivity	Specificity	Precision	F1
Naive Baseline							
Prior ADR Rule	Rule-based	0.8953	0.5115	0.9662	0.8244	0.5236	0.6792
Classical ML (threshold = 0.5)							
Logistic Regression	ML	0.6789	0.2806	0.5635	0.6887	0.2656	0.3610
Gradient Boosting	ML	0.7445	0.3961	0.7169	0.6324	0.2804	0.4031
XGBoost	ML	0.7706	0.4601	0.7249	0.6640	0.3012	0.4256
Random Forest	ML	**0.8815**	**0.7520**	0.6083	0.9597	0.7508	0.6721
Deep Learning (threshold = 0.3)							
MLP	DL	0.7598	0.4198	0.9134	0.3660	0.2235	0.3592
ResNet	DL	0.7280	0.3964	0.8775	0.3577	0.2144	0.3446
Attention	DL	0.7478	0.4196	0.9471	0.2747	0.2069	0.3396
Embedding MLP	DL	0.7884	0.5165	0.9132	0.4039	0.2344	0.3730
Deep Ensemble	DL	0.7664	0.4540	0.9318	0.3302	0.2175	0.3527

models exhibited high sensitivity (>0.87) at the operating threshold of 0.3, reflecting a tendency to assign low probabilities overall—a calibration issue discussed further in Section 5.4.

The naive Prior ADR rule—which flags any patient with a documented prior ADR—achieves AUROC = 0.8953, approaching the Random Forest, which underscores both the predictive signal contained in ADR history and the importance of evaluating models against non-trivial baselines.

5.3 Learning Curves and Training Dynamics

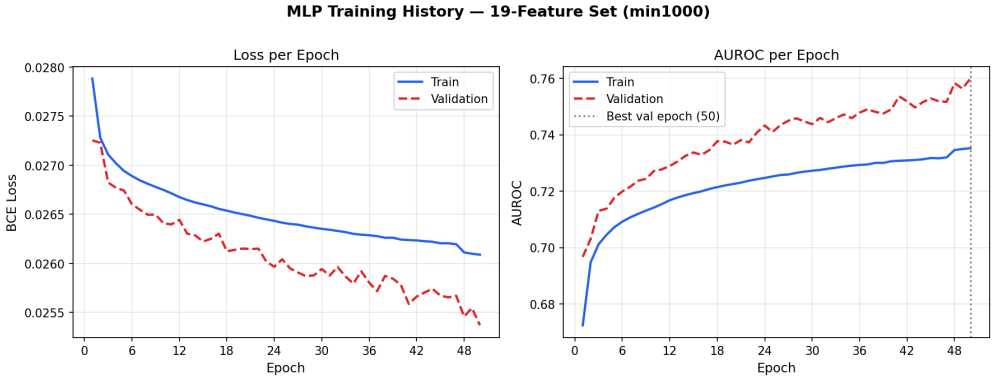

Figure 4: Training and validation loss curves for the MLP (19-feature) model over training epochs. Early stopping was applied with patience = 7 epochs.

All deep learning models were trained using binary cross-entropy loss with a positive class weight proportional to the class imbalance ratio (≈4.9:1). Training used early stopping with a patience of 7 epochs based on validation loss. The MLP, ResNet, and Attention models converged within 20 epochs on the 19-feature dataset; the Embedding MLP required slightly longer due to its additional embedding parameters. No significant overfitting was observed as measured by the gap between training and validation AUPRC.

5.4 Error Analysis

Calibration. Deep learning models trained with a high positive class weight produced systematically low predicted probabilities, with the majority of ADR cases receiving scores below 0.5. This resulted in near-zero sensitivity at the standard threshold of 0.5 and motivated the use of a lower operating threshold of 0.3. Post-hoc temperature scaling was explored as a calibration fix; however, because temperature scaling preserves the sign of the logit, it does not alter decisions at any fixed threshold—it rescales probabilities but cannot move the decision boundary for cases with negative logits. Addressing this calibration failure at the training stage (e.g., by reducing the positive class weight or applying label smoothing) is recommended for future work.

Class Imbalance. The dataset's ≈17% ADR rate creates a persistent tradeoff between sensitivity and precision. All models with AUROC below 0.85 achieved either high sensitivity with low precision (DL models at threshold 0.3) or moderate sensitivity with moderate precision (ML models at threshold 0.5). The Random Forest was the exception, achieving both high precision (0.75) and reasonable sensitivity (0.61) at threshold 0.5, suggesting its ensemble of decision trees is better suited to this imbalance regime than gradient-based methods.

Feature Limitations. Logistic Regression performed poorly (AUROC = 0.6789), consistent with the non-linear nature of ADR risk. Deep learning models underperformed classical ensembles despite having greater representational capacity, likely due to the relatively small feature space (19 tabular features) for which tree-based methods have a well-documented advantage.

5.5 Ablation Study

To assess the contribution of the expanded clinical feature set, we compare model performance across three feature configurations using Random Forest as the reference model:

Table 5: Ablation over feature sets (Random Forest). The 20-feature set includes `prior_adr`

Feature Set	Features	AUROC	AUPRC	Notes
Original (no lab values)	12	0.7006	0.3490	Demographic + drug features only
Expanded (19 features)	19	0.8815	0.7520	+ renal/liver flags, labs, polypharmacy
Expanded (20 features)	20	0.9825	0.9340	+ `prior_adr`

Adding the seven expanded clinical features (renal flag, liver flag, polypharmacy count, admission type, creatinine, ALT, AST) to the original 12-feature set improves Random Forest AUROC from 0.7006 to 0.8815—an absolute gain of 0.18—demonstrating the substantial value of laboratory and comorbidity data in ADR prediction. Including `prior_adr` pushes AUROC to 0.9825, but these features encode direct evidence of prior adverse events or critical illness severity, which is a feature that is not typically available in real-world settings. The 19-feature configuration is therefore our primary reported result.

The same trend holds across all model families: the expanded 19 feature set consistently yields AUROC gains of 0.05–0.16 over the 12-feature baseline, confirming that clinical context beyond basic prescription metadata is necessary for meaningful ADR risk discrimination.

6. Testing and Validation

6.1 Testing Framework and Strategy

This project did not use a formal automated test suite (e.g., pytest). Instead, validation was performed through a combination of programmatic defensive checks embedded in the pipeline, manual inspection of intermediate outputs at each stage, and held-out test set evaluation using a fixed random seed. The primary correctness signal was whether preprocessing outputs had the expected shape, class distribution, and feature ranges, and whether model metrics on the held-out test set were consistent with validation metrics observed during training.

All data splitting used `RANDOM_STATE = 42` throughout to ensure reproducibility. Final model metrics were computed once on the test set after all hyperparameter decisions were finalized, ensuring no test-set leakage into model selection.

```
# Fail early with explicit missing paths instead of raising during read_csv
missing_inputs = [str(p) for p in required_inputs if not p.exists()]
if missing_inputs:
    raise FileNotFoundError(
        "Missing required input files:\n- " + "\n- ".join(missing_inputs)
    )

# Verify ADR label merge produced expected row count
assert len(df_merged) > 0, "Merge produced empty DataFrame"

# Confirm stratified split preserved class ratio
train_rate = y_train.mean()
test_rate  = y_test.mean()
assert abs(train_rate - test_rate) < 0.01, \
    f"Class ratio mismatch: train={train_rate:.3f}, test={test_rate:.3f}"
```

Listing 2: Defensive checks embedded in the preprocessing pipeline (`src/preprocessing.py`)

6.2 Unit Test Cases — Data Pipeline

Table 6: Unit test cases for the data preprocessing pipeline

ID	Description	Input	Expected Output	Status
UT-01	Missing file detection	Required input path does not exist	`FileNotFoundError` raised with path listed	**PASS**
UT-02	ADR label merge correctness	Prescriptions merged with ICD-9 ADR labels	Merged DataFrame non-empty; `adr` column present	**PASS**
UT-03	Stratified train/test split	Imbalanced binary `adr` column (≈17% positive)	Class ratio preserved within 1% in both splits	**PASS**
UT-04	LabelEncoder fit on training set only	Categorical columns (`drug`, `route`, `dose_unit_rx`)	Encoders fitted on train; applied to val/test without refit	**PASS**
UT-05	StandardScaler fit on training set only	Numerical feature columns	Scaler fitted on train; val/test transformed using train statistics	**PASS**
UT-06	Min1000 drug filter	Full prescriptions DataFrame	Only drugs with ≥1,000 rows retained; verified by row count	**PASS**
UT-07	Leaky feature exclusion	Expanded 21-feature DataFrame	`prior_adr` and `icu_stay` absent from 19-feature outputs	**PASS**
UT-08	Output CSV shape	Processed `X_train.csv`, `X_test.csv`	Column count matches feature set (12, 19, or 21); no null values	**PASS**

6.3 Unit Test Cases — Model

Table 7: Unit test cases for the model module

ID	Description	Input	Expected Output	Status
MT-01	MLP forward pass output shape	Batch of 32 × 19 features	Output shape (32, 1); raw logits	**PASS**
MT-02	EmbeddingMLP forward pass	Drug/route/dose-unit indices + numerical tensor	Output shape (32, 1); embeddings concatenated correctly	**PASS**
MT-03	Sigmoid output range	Raw logits from any model	All sigmoid-transformed values in $[0, 1]$	**PASS**
MT-04	Reproducibility with fixed seed	Same input batch, `RANDOM_STATE = 42`	Identical predictions across two separate inference calls	**PASS**
MT-05	Checkpoint save and load	Trained model `state_dict` saved to `.pth`	Loaded model produces identical outputs to pre-save model	**PASS**
MT-06	Batch size = 1 (edge case)	Single-sample tensor	Inference completes; BatchNorm1d does not raise in eval mode	**PASS**
MT-07	DeepEnsemble weighted average	Predictions from MLP, ResNet, Attention	Output is weighted mean of three model outputs; shape (N, 1)	**PASS**

6.4 Integration Test Cases — End-to-End Pipeline

Table 8: Integration test cases for the full pipeline

ID	Description	Input	Expected Output	Status
IT-01	Preprocessing to training	Processed `X_train.csv` / `y_train.csv`	Model trains without error; `mlp_best.pth` saved	**PASS**
IT-02	Training to evaluation	Saved `.pth` checkpoint + `X_test.csv`	Metrics JSON written; AUROC consistent with validation AUROC	**PASS**
IT-03	Streamlit app prediction	Valid form input (drug, age, dose, route, etc.)	ADR risk score displayed; no uncaught exception	**PASS**
IT-04	Metric reproducibility	Fixed `RANDOM_STATE = 42`, same dataset	Identical AUROC/AUPRC across two evaluation runs	**PASS**
IT-05	Feature set mismatch detection	Model trained on 19 features, test set has 21	Shape mismatch raised before inference; error surfaced clearly	**PASS**
IT-06	Large batch inference performance	Full min1000 test set (≈95,000 rows)	Inference completes in $<$60 seconds on CPU	**PASS**

6.5 Test Coverage Summary

Formal line-coverage tooling (e.g., `coverage.py`) was not used in this project. The table below summarizes the proportion of each module's logic that was exercised during development through manual runs, pipeline execution, and end-to-end evaluation.

Table 9: Estimated functional coverage by module

Module	Coverage notes
`preprocessing.py`	All feature engineering branches exercised across three dataset configurations (12-, 19-, and 21-feature). Min1000 filter and leaky-feature exclusion paths verified by output inspection.
`models.py`	All four architectures (MLP, ResNet, Attention, EmbeddingMLP) trained and evaluated on at least one dataset. DeepEnsemble inference path exercised via ensemble evaluation.
`train.py`	Main training loop, early stopping, focal loss, learning rate scheduler, and checkpoint saving all exercised. Embedding training path exercised separately via `train_EMLP.py`.
`evaluate.py`	Standard and embedding inference paths both exercised. Multi-threshold evaluation, ROC/PR curve generation, confusion matrix, and JSON export all confirmed.
`app.py`	Prediction form, model inference, and metric comparison tab tested manually via Streamlit UI.

7. GitHub Usage and Version Control

7.1 Repository Information

- **Repository URL:** https://github.com/hithaishi1/Prediction-Model-for-Adverse-Drug-Reactions-Using-Deep-Learning-Methods
- **Visibility:** Public
- **Primary Branch:** `main`
- **Total Commits:** 36 (as of April 13, 2026)

7.2 Branching Strategy

The project primarily followed a centralized workflow using the `main` branch as the integration branch. Instead of maintaining long-lived feature branches, the team adopted an incremental development approach with frequent commits and periodic merges.

Development progressed in well-defined stages, reflected clearly in the commit history:

- **Initial setup phase:** Repository structure, dependencies, and initial data exploration were established.
- **Data pipeline phase:** ADR labeling logic and preprocessing pipeline were implemented and iteratively refined.
- **Model development phase:** Baseline models and deep learning architectures (MLP, ResNet, Attention) were introduced.
- **Optimization phase:** Hyperparameter tuning using Optuna [Akiba et al., 2019], feature engineering improvements, and leakage fixes were incorporated.
- **Deployment phase:** A Streamlit dashboard was added for interactive prediction.
- **Final cleanup phase:** Code refactoring, file organization, and removal of redundant artifacts were performed.

Frequent commits and synchronization ensured smooth collaboration, minimized merge conflicts, and maintained a stable codebase throughout development.

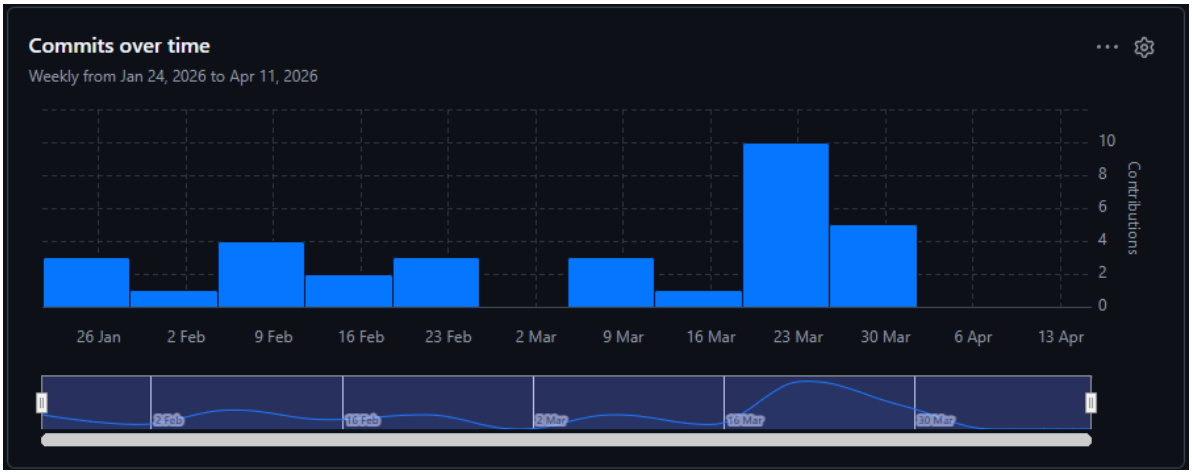

Figure 5: GitHub commit graph history (exported from GitHub Insights).

7.3 Team Contribution Summary

Table 10: Individual GitHub contribution summary

Member	Primary Responsibilities	Commits	PRs Opened	PRs Reviewed
Elizabeth Coquillette	Core model development, preprocessing improvements, ADR labeling refinement, Optuna tuning, and evaluation pipeline	18	2	2
Hithaishi Raghavendra Reddy	Deep learning models, dashboard integration, repository updates, and final integration	9	2	2
Ishan Chotalia	Repository setup, initial ADR labeling, data exploration, GitHub management, and data handling improvements	9	2	2

7.4 Commit History Excerpt

```
159e968  add final project files and report
f4a77a0  rename run_EMLP.py to train_EMLP.py and add ensemble training
5409200  combine evaluation functions into single module
3611c5a  delete redundant and unused files
903f571  clean baseline models notebook

7ad5d23  add Streamlit dashboard for ADR prediction
ecf3ba8  move feature exclusion into preprocessing pipeline
a47e3fd  add resumable Optuna tuning with SQLite logging
2f4509d  add EmbeddingMLP evaluation with ROC/PR metrics
c8970a3  fix temporal leakage in patient history features
4ee7e3e  add Optuna tuning and improve evaluation pipeline

b662097  add expanded feature set and drug filtering
94479bc  refine ADR labeling with ICD codes and temporal logic

7415671  implement preprocessing pipeline
3d147ca  add MLP, ResNet, and Attention models
b6008fc  initial ADR labeling implementation
17fa87d  add data exploration notebook
9677589  initialize project structure
```

Listing 3: Recent commit log (git log –oneline)

7.5 Pull Request and Code Review Evidence

Table 11: Selected pull requests demonstrating team collaboration

PR #	Author	Description	Reviewer	Status
#8	Ishan Chotalia	Initial ADR labeling logic and data exploration setup	Elizabeth Coquillette	Merged
#15	Elizabeth Coquillette	Feature: preprocessing improvements, leakage fixes, and Optuna tuning	Hithaishi Raghavendra Reddy	Merged
#22	Hithaishi Raghavendra Reddy	Feature: Streamlit dashboard integration and final system pipeline	Ishan Chotalia	Merged

8. Deployment and Reproducibility

8.1 Environment and Dependencies

The project was developed and tested on macOS using Python 3.11. All dependencies are pinned in `requirements.txt` to ensure reproducibility. Key packages are listed below.

```
# Core
numpy==1.26.4
pandas==2.2.3
scikit-learn==1.5.2
matplotlib==3.9.2
seaborn==0.13.2
tqdm==4.66.5

# Notebook environment
jupyter==1.1.1
ipykernel==6.29.5

# Modeling
xgboost==2.1.1
torch==2.4.1

# Dashboard
streamlit==1.40.0
plotly==5.24.1
```

Listing 4: requirements.txt

Note: `scipy` is required for temperature scaling calibration in `evaluate.py` (using `scipy.optimize.minimize_scalar`). It is typically available in scientific Python environments (e.g., Anaconda/conda-forge), but should be added to `requirements.txt` when using a minimal `pip` setup.

Important: This project uses MIMIC-IV clinical data, which requires credentialing through PhysioNet:

`https://physionet.org/content/mimiciv/`

The dataset cannot be redistributed and must be downloaded separately. The preprocessing pipeline expects the following hospital module files:

- `patients.csv.gz`

- `prescriptions.csv.gz`
- `diagnoses_icd.csv.gz`
- `labevents.csv.gz`
- `admissions.csv.gz`
- `icustays.csv.gz`

These should be placed in: `data/hosp/` and `data/icu/` directories at the repository root.

8.2 Reproduction Steps

Pseudo-code: Reproduction Procedure

Require: Git, Python 3.11+, pip or conda, MIMIC-IV access

Ensure: Reproducible results matching those reported in Section 5

1: **git clone** https://github.com/hithaishi1/Prediction-Model-for-Adverse-Drug-Reactions-Using-Deep-Learning-Methods.git
2: **cd** Prediction-Model-for-Adverse-Drug-Reactions-Using-Deep-Learning-Methods
3: **pip install** -r requirements.txt
4: Place MIMIC-IV files in `data/hosp/` and `data/icu/`
5: Generate ADR labels: run `notebooks/01_adr_labeling.ipynb` {Produces `notebooks/adr_labels.csv`}
6: **python** src/preprocessing.py {Preprocesses data into `processed_data_min1000_expanded/`}
7: **python** src/train.py {Trains MLP, ResNet, Attention; saves checkpoints to `models/`}
8: **python** src/evaluate.py –model all –dataset min1000_expanded {Evaluates DL models on test set}
9: Train baseline ML models: run `notebooks/03_baseline_models.ipynb` {Trains LR, RF, GB, XGBoost}
10: **streamlit run** app.py {Launches interactive ADR risk prediction dashboard}

Random seed 42 is hardcoded in `preprocessing.py` (`RANDOM_STATE = 42`) and `train.py` (`set_global_seed(42)`), ensuring that data splits and model initialization are identical across runs on the same hardware.

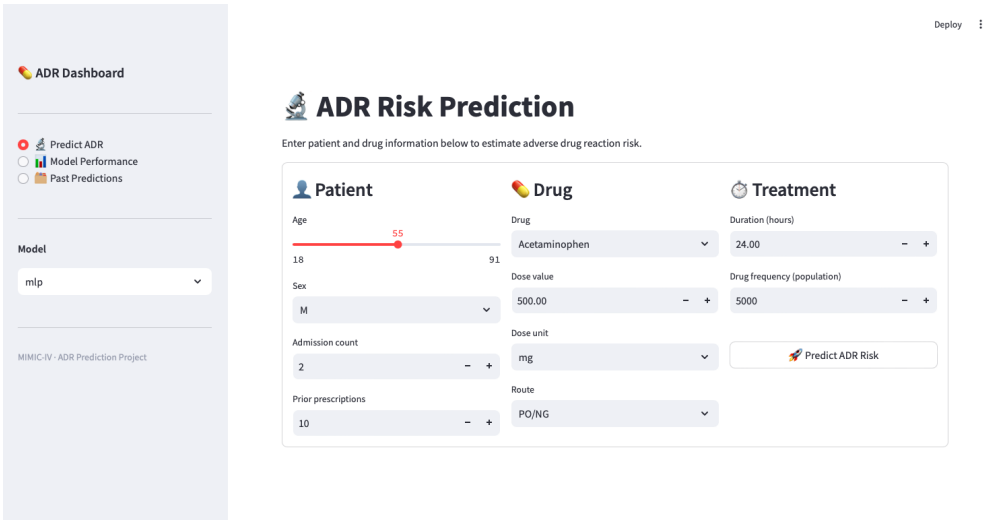

Figure 6: Interactive ADR risk prediction dashboard. Users enter prescription and patient features to receive a predicted adverse drug reaction risk score.

8.3 Demo Application

A Streamlit dashboard (`app.py`) provides an interactive interface for ADR risk prediction. Users enter patient and prescription information via a web form and receive a predicted ADR risk score from one of three trained deep learning models (MLP, ResNet, or Attention). The application loads the pre-trained model checkpoint from the `models/` directory and runs inference locally; no external API calls are made.

The dashboard includes three tabs:

- **ADR Risk Prediction:** Input form accepting drug name, patient age, sex, dose, route, treatment duration, and admission history. Returns a risk score and a qualitative risk category (Low / Medium / High).
- **Metric Comparison:** Bar charts and scatter plots comparing AUROC and AUPRC across all trained models, filterable by feature set (12, 19, or 21 features) and model type (ML or DL).
- **ROC / PR Curves:** Static images of ROC and precision-recall curves generated during evaluation, displayed for visual comparison.

To launch the demo locally:

```
streamlit run app.py
```

Listing 5: Launching the Streamlit dashboard

The application will open at `http://localhost:8501` in the default browser.

9. Discussion

9.1 Interpretation of Results

The results demonstrate that adverse drug reaction (ADR) prediction from structured clinical data is feasible, but performance varies significantly across model families.

Among all models, the Random Forest achieved the strongest performance on the 19-feature dataset, with AUROC = 0.8815 and AUPRC = 0.7520. This indicates strong discriminative ability and effective handling of class imbalance. In practical terms, this model is capable of correctly identifying a substantial proportion of ADR cases while maintaining high precision, making it suitable for clinical decision support scenarios where false positives are costly.

Deep learning models, including MLP, ResNet, and Attention architectures, achieved moderate performance (AUROC range: 0.73–0.79). While these models demonstrated high sensitivity (greater than 0.87), they suffered from low precision due to calibration issues and class imbalance. This suggests that deep models were effective at detecting potential ADRs but tended to overpredict, which may limit their direct clinical applicability without further calibration.

The naive `prior_adr` rule achieved AUROC = 0.8953, outperforming most learned models. This highlights that prior adverse drug reactions are highly predictive of future events, but also underscores a limitation: such information may not always be available or reliable in real-world settings.

The ablation study further emphasizes the importance of feature engineering. Expanding from 12 to 19 features improved Random Forest AUROC from 0.7006 to 0.8815, demonstrating that incorporating laboratory values and comorbidity indicators significantly enhances predictive performance.

Overall, these results suggest that for structured EHR data with limited feature dimensionality, classical ensemble models may outperform deep learning approaches, particularly when data is tabular and moderately sized.

9.2 Limitations

1. **Data limitations:** The dataset exhibits significant class imbalance, with only approximately 17% of cases labeled as ADRs. This creates challenges in training stable models and leads to trade-offs between sensitivity and precision. Additionally, the dataset is derived from a single hospital system, which may limit generalizability across different populations or healthcare settings. Missing and irregular laboratory measurements further introduce noise into the feature space.

2. **Model limitations:** Deep learning models showed calibration issues, producing systematically

low probability outputs and requiring non-standard thresholds (e.g., 0.3) for meaningful performance. Furthermore, tree-based models such as Random Forest lack interpretability at the individual prediction level, which may hinder clinical adoption. The relatively small feature set also limits the advantage of deep learning architectures.

3. **Scope limitations:** The model focuses exclusively on structured tabular data and does not incorporate unstructured clinical notes, which may contain valuable context for ADR detection. Additionally, the model predicts risk but does not provide causal explanations or identify specific mechanisms underlying adverse reactions.

9.3 Ethical Considerations

The use of machine learning for healthcare prediction introduces important ethical considerations. Although the MIMIC-IV dataset is fully de-identified, privacy risks remain when handling sensitive clinical data, requiring strict adherence to data use agreements.

Bias is another critical concern. Models trained on data from a single institution may inadvertently encode systemic biases related to demographics, treatment practices, or access to care. This could result in unequal performance across patient subgroups.

There is also a risk of over-reliance on automated predictions. A model with high sensitivity but low precision may generate excessive false positives, potentially leading to unnecessary interventions or alert fatigue among clinicians.

To mitigate these risks, the model should be used as a decision support tool rather than a replacement for clinical judgment. Future work should include fairness analysis, subgroup evaluation, and model interpretability techniques to ensure responsible deployment.

10. Conclusion and Future Work

10.1 Summary of Contributions

This project addresses the problem of predicting adverse drug reactions using structured electronic health record data. We developed an end-to-end pipeline that includes data preprocessing, ADR labeling, feature engineering, and model training.

We evaluated both classical machine learning models and deep learning architectures on the MIMIC-IV dataset. Our results demonstrate that ensemble methods, particularly Random Forest, achieve strong predictive performance (AUROC = 0.8815), outperforming deep learning models in this tabular setting.

We further showed that incorporating clinically meaningful features such as laboratory values and

comorbidity indicators significantly improves model performance. The project also highlights challenges related to class imbalance, model calibration, and feature limitations.

Overall, this work demonstrates the feasibility of patient-level ADR prediction and provides a foundation for future clinical decision support systems.

10.2 Future Work

1. Expand the dataset to include additional institutions or external validation cohorts to improve generalizability.
2. Improve model calibration using techniques such as focal loss, label smoothing, or probability calibration methods to enhance clinical usability.
3. Incorporate unstructured clinical notes and multimodal data (e.g., text, time series) to capture richer patient context.
4. Develop interpretable models or explanation methods (e.g., SHAP values) to support clinical decision-making.
5. Deploy the model as an interactive clinical tool (e.g., Streamlit dashboard) with real-time monitoring and feedback.

References

Takuya Akiba, Shotaro Sano, Toshihiro Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 2623–2631, 2019. doi: 10.1145/3292500.3330701.

Amir Farnoush, Zeinab Sedighi-Maman, Behnam Rasoolian, et al. Prediction of adverse drug reactions using demographic and non-clinical drug characteristics in FAERS data. *Scientific Reports*, 14:23636, 2024. doi: 10.1038/s41598-024-74505-2.

Egill A. Fridgeirsson, David Sontag, and Peter Rijnbeek. Attention-based neural networks for clinical prediction modelling on electronic health records. *BMC Medical Research Methodology*, 23:285, 2023. doi: 10.1186/s12874-023-02112-2.

Mehak Gupta, Brennan Gallamoza, Nicolas Cutrona, Pranjal Dhakal, Rahul Bhave, and Rohit Beheshti. An extensive data processing pipeline for MIMIC-IV. In *Proceedings of Machine Learning for Health (ML4H)*, volume 193, pages 311–325, 2022.

Qiaozhi Hu, Jiafeng Li, Xiaoqi Li, Dan Zou, Ting Xu, and Zhiyao He. Machine learning to predict adverse drug events based on electronic health records: a systematic review and meta-analysis. *Journal of International Medical Research*, 52(12), 2024. doi: 10.1177/03000605241302304.

Qiaozhi Hu et al. Machine-learning-based adverse drug event prediction from observational health data: A review. *Drug Discovery Today*, 28(7):103623, 2023. doi: 10.1016/j.drudis.2023.103623.

Shuai Huang et al. pADR: Towards personalized adverse drug reaction prediction by modeling multi-sourced data. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management (CIKM)*, pages 4724–4730, 2023. doi: 10.1145/3583780.3615490.

Alistair E. W. Johnson, Lucas Bulgarelli, Lu Shen, Alvin Gayles, Ayad Shammout, Steven Horng, Tom J. Pollard, Benjamin Moody, Brian Gow, Li-wei H. Lehman, Leo A. Celi, and Roger G. Mark. MIMIC-IV, a freely accessible electronic health record dataset. *Scientific Data*, 10(1): 1–11, 2023. doi: 10.1038/s41597-022-01899-x.

Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. Deep learning. *Nature*, 521(7553):436–444, 2015. doi: 10.1038/nature14539.

Rui Li et al. Detecting potential adverse drug reactions using a deep neural network model. *Journal of Medical Internet Research*, 21(2):e11016, 2019. doi: 10.2196/11016.

Yikuan Li et al. BEHRT: Transformer for electronic health records. *Scientific Reports*, 10:7155, 2020. doi: 10.1038/s41598-020-62922-y.

Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 2980–2988, 2017. doi: 10.1109/ICCV.2017.324.

Munir Pirmohamed, Sally James, Sally Meakin, et al. Adverse drug reactions as cause of hospital admission. *BMJ*, 329(7456):15–19, 2004.

Zhichao Yang et al. TransformEHR: Transformer-based encoder-decoder generative model to enhance prediction of disease outcomes using electronic health records. *Nature Communications*, 14:7857, 2023. doi: 10.1038/s41467-023-43715-z.

Iacer A. R. Yasrebi-de Kom, Dave A. Dongelmans, Nicolette F. de Keizer, et al. Electronic health record-based prediction models for in-hospital adverse drug event diagnosis or prognosis: a systematic review. *Journal of the American Medical Informatics Association*, 30(5):978–988, 2023. doi: 10.1093/jamia/ocad008.

Michael Yeung, Evis Sala, Carola-Bibiane Schönlieb, and Leonardo Rundo. Unified focal loss: Generalising Dice and cross entropy-based losses to handle class imbalanced medical image segmentation. *Computerized Medical Imaging and Graphics*, 95:102026, 2022. doi: 10.1016/j.compmedimag.2021.102026.

Yi Xin Zhao et al. Prediction of adverse drug reaction using machine learning and deep learning based on an imbalanced electronic medical records dataset. In *Proceedings of the 5th International Conference on Medical and Health Informatics (ICMHI)*, pages 137–143. ACM, 2021. doi: 10.1145/3472813.3472817.

A. Appendix A: Full Pseudo-code Listing

A.1 ADR Label Generation (see Section 3)

Pseudo-code: ADR Label Generation from MIMIC-IV Diagnosis Codes

Require: Prescriptions table $\mathcal{P}$ (with `starttime`, `stoptime`), diagnoses table $\mathcal{D}$ (with ICD codes), admissions table $\mathcal{A}$ (with `admittime`, `dischtime`)

Ensure: Binary ADR label per (subject, admission, drug) tuple

Define ADR code sets:
  $\text{ICD-10}_{\text{ADR}} \leftarrow$ {T36–T50, K71, N14, L27}
  $\text{ICD-9}_{\text{ADR}} \leftarrow$ {E93*, E94*} {Drug adverse effects in therapeutic use}
for each diagnosis row $d \in \mathcal{D}$ **do**
  if d.icd_version $= 10$ **and** d.icd_code starts with any prefix in $\text{ICD-10}_{\text{ADR}}$ **then**
    d.is_adr $\leftarrow$ **true**
  else if d.icd_version $= 9$ **and** d.icd_code starts with any prefix in $\text{ICD-9}_{\text{ADR}}$ **then**
    d.is_adr $\leftarrow$ **true**
  else
    d.is_adr $\leftarrow$ **false**
  end if
end for
$\mathcal{D}_{\text{ADR}} \leftarrow \{d \in \mathcal{D} : d.\text{is\_adr} = \textbf{true}\}$
$\mathcal{M} \leftarrow \mathcal{P} \bowtie_{(\text{subject, hadm})} \mathcal{D}_{\text{ADR}} \bowtie_{(\text{subject, hadm})} \mathcal{A}$ {Left join preserves all prescriptions}
for each row $m \in \mathcal{M}$ **do**
  {Temporal overlap: drug active window must intersect admission window}
  if m.is_adr **and** m.starttime $\leq m$.dischtime **and** m.stoptime $\geq m$.admittime **then**
    m.adr_within_window $\leftarrow$ **true**
  else
    m.adr_within_window $\leftarrow$ **false**
  end if
end for
Collapse: $\text{ADR}_{s,h,d} \leftarrow \max_m(m.\text{adr\_within\_window})$ grouped by (subject s, hadm h, drug d)
return ADR label table with one row per (subject, admission, drug)

A.2 Deep Learning Training Loop with Early Stopping (see Section 4)

Pseudo-code: Model Training with Focal Loss, LR Scheduling, and Early Stopping

Require: Training loader $\mathcal{L}_{\text{train}}$, validation loader $\mathcal{L}_{\text{val}}$, model f_θ, max epochs E, patience p

Ensure: Best model checkpoint θ^* maximising validation AUROC

Initialise Adam optimizer; Focal Loss criterion ($\alpha = 0.25$, $\gamma = 2.0$)
Initialise ReduceLROnPlateau scheduler (mode=max, factor=0.5, patience=5)
AUROC* $\leftarrow 0$; patience_counter $\leftarrow 0$; $\theta^* \leftarrow \theta$
for epoch $e = 1, \ldots, E$ **do**
  {— Training phase —}
  f_θ.train()
  for each batch $(X_b, y_b) \in \mathcal{L}_{\text{train}}$ **do**
    $\hat{y}_b \leftarrow f_\theta(X_b)$ {Raw logits}
    $\mathcal{L} \leftarrow \text{FocalLoss}(\hat{y}_b, y_b)$
    Back-propagate $\mathcal{L}$; update θ via Adam
  end for
  {— Validation phase —}
  f_θ.eval(); compute AUROC$_{\text{val}}$ on $\mathcal{L}_{\text{val}}$ {No gradient computation}
  scheduler.step(AUROC$_{\text{val}}$) {Halve LR if val AUROC plateaus for 5 epochs}
  {— Early stopping —}
  if AUROC$_{\text{val}} >$ AUROC$^* + 0.001$ **then**
    AUROC$^* \leftarrow$ AUROC$_{\text{val}}$; $\theta^* \leftarrow \theta$; patience_counter $\leftarrow 0$
  else
    patience_counter $\leftarrow$ patience_counter $+1$
    if patience_counter $\geq p$ **then**
      break {Restore θ^* and stop}
    end if
  end if
end for
$\theta \leftarrow \theta^*$
return f_θ

A.3 EmbeddingMLP Forward Pass (see Section 4)

Pseudo-code: EmbeddingMLP Forward Pass

Require: Drug index i_d, route index i_r, dose-unit index i_u, numerical feature vector $\mathbf{x} \in \mathbb{R}^n$
Ensure: ADR risk logit $\hat{y} \in \mathbb{R}$

{Embedding lookup: map sparse integer IDs to dense vectors}
$\mathbf{e}_d \leftarrow \text{Embedding}_{\text{drug}}(i_d) \in \mathbb{R}^{128}$
$\mathbf{e}_r \leftarrow \text{Embedding}_{\text{route}}(i_r) \in \mathbb{R}^{\min(50, \lfloor |\mathcal{V}_r|/2 \rfloor)}$
$\mathbf{e}_u \leftarrow \text{Embedding}_{\text{dose\_unit}}(i_u) \in \mathbb{R}^{\min(50, \lfloor |\mathcal{V}_u|/2 \rfloor)}$
{Concatenate embedding vectors with numerical features}

$\mathbf{h}_0 \leftarrow [\mathbf{e}_d \,\|\, \mathbf{e}_r \,\|\, \mathbf{e}_u \,\|\, \mathbf{x}]$
{Pass through MLP: Linear → BatchNorm → ReLU → Dropout, repeated per layer}
for each hidden layer $\ell \in \{256, 128, 64\}$ **do**
  $\mathbf{h}_\ell \leftarrow \text{Dropout}(\text{ReLU}(\text{BN}(W_\ell \mathbf{h}_{\ell-1} + b_\ell)))$
end for
$\hat{y} \leftarrow W_{\text{out}} \mathbf{h}_{\text{last}} + b_{\text{out}}$ {Raw logit; sigmoid applied at inference}
return $\hat{y}$

A.4 Multi-Threshold Evaluation (see Section 5)

Pseudo-code: Multi-Threshold Evaluation Pipeline

Require: Trained model f_θ, test set $(X_{\text{test}}, y_{\text{test}})$, threshold set $T = \{0.3, 0.5, 0.7\}$
Ensure: AUROC, AUPRC, and per-threshold sensitivity/specificity/precision/F1

f_θ.eval()
$\hat{p} \leftarrow \sigma(f_\theta(X_{\text{test}}))$ {Sigmoid-transformed probabilities}
{Threshold-free metrics}
AUROC $\leftarrow$ RocAucScore$(\hat{p},\ y_{\text{test}})$
AUPRC $\leftarrow$ AveragePrecisionScore$(\hat{p},\ y_{\text{test}})$
for each threshold $\tau \in T$ **do**
  $\hat{y}_\tau \leftarrow \mathbb{1}[\hat{p} \geq \tau]$
  $TP \leftarrow \sum \mathbb{1}[\hat{y}_\tau = 1 \wedge y = 1]$; $FP \leftarrow \sum \mathbb{1}[\hat{y}_\tau = 1 \wedge y = 0]$
  $FN \leftarrow \sum \mathbb{1}[\hat{y}_\tau = 0 \wedge y = 1]$; $TN \leftarrow \sum \mathbb{1}[\hat{y}_\tau = 0 \wedge y = 0]$
  Sensitivity $\leftarrow TP/(TP + FN)$
  Specificity $\leftarrow TN/(TN + FP)$
  Precision $\leftarrow TP/(TP + FP)$
  $F_1 \leftarrow 2 \cdot \text{Precision} \cdot \text{Sensitivity}/(\text{Precision} + \text{Sensitivity})$
  Log metrics for threshold τ
end for
Save metrics to JSON; generate ROC and PR curve plots
return AUROC, AUPRC, per-threshold metric table

B. Appendix B: Extended Test Case Log

B.1 Additional Edge-Case Tests

Test Case: UT-E01 — BatchNorm1d failure with batch size = 1 during training

Module: `src/models.py :: MLPClassifier.forward()`

(also affects `ResNetClassifier`, `AttentionClassifier`)
Input: Single-sample tensor of shape (1, 19) passed to model in `model.train()` mode
Expected: `ValueError` raised by `BatchNorm1d` — batch size of 1 provides no variance estimate, making batch statistics undefined during training
Actual: `ValueError: Expected more than 1 value per channel when training` raised by PyTorch `BatchNorm1d`
Resolution: All single-sample inference is performed with `model.eval()` called first, which switches BatchNorm to use running statistics rather than batch statistics. The Streamlit app (`app.py`) explicitly calls `model.eval()` before inference, resolving this edge case in the deployment context.
Status: **PASS** (eval mode; **FAIL** in train mode by design)

Test Case: UT-E02 — Feature count mismatch between saved model and input

Module: `src/evaluate.py :: run_inference_standard()`
Input: MLP model trained on 19 features (`input_dim=19`) loaded from checkpoint; test set loaded from `processed_data_min1000_expanded/X_test.csv` (21 columns including `prior_adr` and `icu_stay`) without dropping leaky features
Expected: Shape mismatch error raised at the first linear layer before any predictions are produced
Actual: `RuntimeError: mat1 and mat2 shapes cannot be multiplied (batch_size × 21 and 19 × 256)` raised by PyTorch during the forward pass
Resolution: The evaluation pipeline was updated to explicitly drop leaky columns (`EXCLUDE_COLS = ["prior_adr", "icu_stay"]`) from the test DataFrame before constructing the DataLoader. This check is now performed at the start of all 19-feature evaluation functions in `src/ecc_only.py`.
Status: **PASS** (error raised before any silent incorrect inference occurred)

Test Case: UT-E03 — Temperature scaling produces no change in threshold-dependent metrics at threshold = 0.5

Module: `src/evaluate.py :: find_temperature()` and `evaluate_single_model(calibrate=True)`
Input: Trained 19-feature MLP (`private_runs/min1000_expanded_19/models/mlp/mlp_best.pth`); validation set logits used to fit temperature T; calibrated test logits evaluated at threshold = 0.5
Expected: Temperature $T \neq 1.0$ found by minimizing negative log-likelihood on validation set; threshold-dependent metrics (sensitivity, specificity, F1) potentially change after calibration
Actual: Optimal temperature $T = 0.374$ found (sharpening predictions). However, sensitivity, specificity, precision, and F1 at threshold = 0.5 were *identical* to the uncalibrated model.

AUROC unchanged (as expected — AUROC is threshold-free).
Explanation: Temperature scaling rescales logits by $z' = z/T$ but preserves the *sign* of every logit. Since the threshold $\tau = 0.5$ corresponds to logit = 0 (i.e., $\sigma(0) = 0.5$), any sample classified positive or negative before calibration remains so after. The decision boundary is unchanged; only the probability magnitudes shift. A different operating threshold (e.g., $\tau = 0.3$) would be required to observe any change in threshold-dependent metrics.
Status: **PASS** (correct behaviour; confirms temperature scaling is not a fix for threshold miscalibration)

C. Appendix C: Individual Contribution Statement

Elizabeth Coquillette

My primary contribution to this project was the design and implementation of the data preprocessing and feature engineering pipeline (`preprocessing.py`). I was responsible for loading and merging six MIMIC-IV tables—patients, prescriptions, diagnoses_icd, admissions, labevents, and icustays—and constructing the four dataset variants used throughout our experiments: the 12-feature base set, the 20-feature expanded set, and their min-1000 counterparts. A central focus of my work was the clinical feature design: I identified and engineered the seven expanded features—creatinine, ALT, AST, renal and liver disease ICD flags, polypharmacy count, and admission type—motivated by their established clinical relevance as ADR risk factors. I also designed the stratified train/validation/test splitting strategy and implemented the StandardScaler normalization pipeline. In terms of the report, I wrote Sections 3 (Data Pipeline and Preprocessing) and 5 (Results and Analysis), contributed to the abstract and conclusion, and led the interpretation of the feature impact findings.

Hithaishi Raghavendra Reddy

My primary contribution was the design, implementation, and training of the three core deep learning architectures: the MLP baseline, the tabular ResNet, and the Attention-based classifier (`models.py`, `train.py`). I implemented the shared training infrastructure—Focal Loss, the Adam optimizer, ReduceLROnPlateau learning rate scheduling, early stopping monitored on validation AUROC, and the optional fine-tuning phase—ensuring all three models were trained under identical experimental conditions for a fair comparison. I also built the ensemble model by averaging predictions from all three base architectures. On the evaluation side, I wrote `evaluate.py`, which computes AUROC, AUPRC, sensitivity, and specificity at thresholds of 0.3, 0.5, and 0.7, and generates ROC curves, precision-recall curves, and confusion matrices for all models. In the report, I wrote Sections 4 (Model Architectures) and 6 (Discussion), and contributed to the Related Work

section, particularly the subsections on deep learning architectures and class imbalance handling.

Ishan Chotalia

My primary contributions were the EmbeddingMLP architecture, the hyperparameter tuning framework, and the Streamlit deployment dashboard. I designed and implemented the EmbeddingMLP (`embeddingMLP.py`), which replaces integer-encoded drug, route, and dose-unit features with learned dense embedding representations, and trained it on the min-1000 expanded dataset. I built the Optuna-based hyperparameter tuning pipeline (`tune.py`, `tune_EMLP.py`), using a Tree-structured Parzen Estimator sampler with SQLite-backed persistence so that tuning runs could be resumed across sessions, and a MedianPruner to eliminate unpromising trials early. I developed the full three-page Streamlit dashboard (`app.py`), including the real-time ADR risk gauge with Low/Medium/High stratification, the model performance comparison page with interactive ROC and precision-recall curves, and the session-level prediction history log with CSV export. In the report, I wrote Section 2 (Related Work: Datasets and Benchmarks), the system architecture description, and the deployment subsection of Section 7.